ESPECIFICAÇÃO TÉCNICA & ARQUITETURA SERVERLESS IAC (TERRAFORM + AWS)

Arquitetura Serverless com Lambda, DynamoDB, Cognito e IAM

Autora: Grazielle Nascimento **Repositório:** github.com/GrazielleNascimento/Terraform

Stack: Terraform • AWS Lambda • DynamoDB • Cognito • IAM

CAMADA SERVERLESS	SEGURANÇA & AUTH	PERSISTÊNCIA NOSQL
4 Lambdas CRUD	**Cognito + IAM Policies**	**DynamoDB Table**

01 Visão Geral da Arquitetura Serverless

Este documento especifica a implementação completa de uma arquitetura Serverless na AWS provisionada via Terraform. O projeto estrutura uma API de gerenciamento de dados com microsserviços orientados a eventos, desacoplamento por responsabilidade única e rígido isolamento de segurança baseado no princípio do menor privilégio.

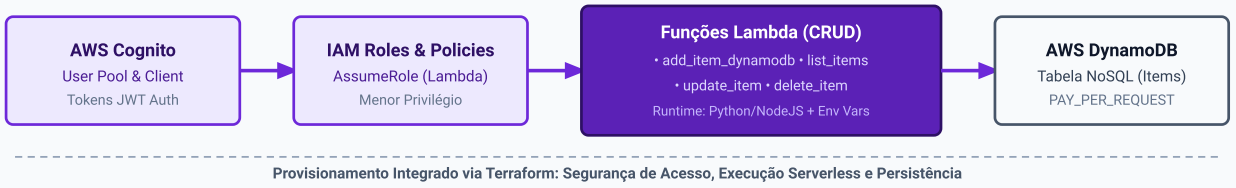


Figura 1: Topologia de infraestrutura da solução Serverless na AWS automatizada com Terraform.

PROPÓSITO E ESCOPO DE ENTREGA

Automatizar via código HCL todo o ecossistema que conecta os diretórios de funções do repositório (`add_item_dynamodb` , `delete_item` , `list_items` e `update_item`) à tabela DynamoDB de destino, garantindo autenticação com Amazon Cognito User Pools e permissões restritas via IAM Roles.

02 Mapeamento de Funções Lambda e Permissões IAM

Cada operação do ciclo de vida dos dados é isolada em um pacote de código específico com sua respectiva função Lambda, garantindo que falhas ou permissões de um endpoint não afetem os demais.

DIRETÓRIO / FUNÇÃO	RECURSO TERRAFORM	AÇÕES IAM MÍNIMAS	FINALIDADE
add_item_dynamodb/	`aws_lambda_function`	`dynamodb:PutItem`	Insere novos registros com IDs únicos e timestamps de criação.
list_items/	`aws_lambda_function`	`dynamodb:Scan` `dynamodb:Query`	Consulta e retorna lista de itens ativos cadastrados na tabela.
update_item/	`aws_lambda_function`	`dynamodb:UpdateItem`	Atualiza propriedades e atributos de um registro existente.
delete_item/	`aws_lambda_function`	`dynamodb:DeleteItem`	Remove logicamente ou fisicamente o item a partir de sua chave.
Observabilidade	`aws_cloudwatch_log_group`	`logs:CreateLogStream` `logs:PutLogEvents`	Geração de logs e métricas de execução para cada função.

03 Estrutura do Repositório e Empacotamento

- **Build Dinâmico com `archive_file`:** O código Terraform utiliza o data source `archive_file` para compactar cada pasta em arquivo `.zip` automaticamente antes do deploy, detectando alterações via `source_code_hash`.
- **Injeção de Variáveis de Ambiente:** O nome da tabela (`TABLE_NAME`) e região AWS são repassados dinamicamente via bloco `environment` para evitar *hardcoding* no código-fonte das funções.
- **Políticas Granulares no IAM:** Criação de papéis de execução dedicados com política `assume_role_policy` para o serviço `lambda.amazonaws.com`, limitando o ARN de acesso exclusivamente à tabela do projeto.

```
      # Exemplo de empacotamento automatizado e injeção de tabela data "archive_file"
"add_item_zip" { type = "zip" source_dir = "${path.module}/add_item_dynamodb" output_path =
"${path.module}/dist/add_item.zip" } resource "aws_lambda_function" "add_item" {
function_name = "add_item_dynamodb" filename = data.archive_file.add_item_zip.output_path
source_code_hash = data.archive_file.add_item_zip.output_base64sha256 role =
aws_iam_role.lambda_exec_role.arn handler = "index.handler" runtime = "nodejs18.x" # ou
python3.11 environment { variables = { TABLE_NAME = aws_dynamodb_table.items_table.name } }
}
```

04 Especificação: Banco de Dados, Autenticação e IAM

Abaixo estão detalhados os recursos essenciais de infraestrutura que dão suporte e garantem a segurança das quatro funções Lambda do projeto.

AWS DynamoDB (`items_table`)

Modo sob demanda (`billing_mode = "PAY_PER_REQUEST"`) para otimização de custo, com chave primária (`hash_key = "id"`), criptografia em repouso com chave AWS gerenciada (KMS) e *Point-in-Time Recovery* ativado.

Amazon Cognito User Pool

Provisionamento do pool de usuários para cadastro e autenticação segura com verificação de e-mail obrigatória, políticas rigorosas de senha e emissão de tokens JWT para chamadas autenticadas.

Cognito App Client

Cliente sem segredo cliente (`generate_secret = false`) para integração com aplicações web/mobile (SPA), com suporte aos fluxos padrão OAuth 2.0 (Authorization Code / SRP).

IAM Roles & Least Privilege

Definição de roles estritas por serviço; nenhum recurso utiliza permissões amplas (`*`). O acesso ao DynamoDB e CloudWatch Logs é restrito aos recursos específicos declarados no Terraform.

05 Roteiro de Tarefas para Execução do Projeto

- **1. Estruturação dos Arquivos HCL:** Criar os arquivos de declaração: `dynamodb.tf` , `cognito.tf` , `iam.tf` , `lambda.tf` , `variables.tf` e `outputs.tf` .
- **2. Padronização dos Handlers Lambda:** Validar o arquivo principal em cada um dos quatro diretórios (`add_item_dynamodb` , `delete_item` , etc.) assegurando o tratamento de erros HTTP (200, 400, 500) e CORS.
- **3. Validação e Execução de Testes:** Executar o ciclo `terraform init` → `terraform validate` → `terraform plan` para verificar a criação ordenada de dependências.
- **4. Documentação das Variáveis e Outputs:** Expor via `outputs.tf` o `user_pool_id` , `user_pool_client_id` , ARNs das Lambdas e o nome da tabela criada.

CHECKLIST DE CONCLUSÃO E ENTREGA

• **Código Modularizado:** Código limpo, formatado com `terraform fmt` e sem credenciais versionadas.
• **Teste Funcional:** Realizar autenticação no Cognito, obter o token JWT e invocar os 4 métodos CRUD comprovando leitura e escrita na tabela DynamoDB.
• **Destruição Limpa:** Garantir que `terraform destroy` remova todos os recursos sem órfãos.